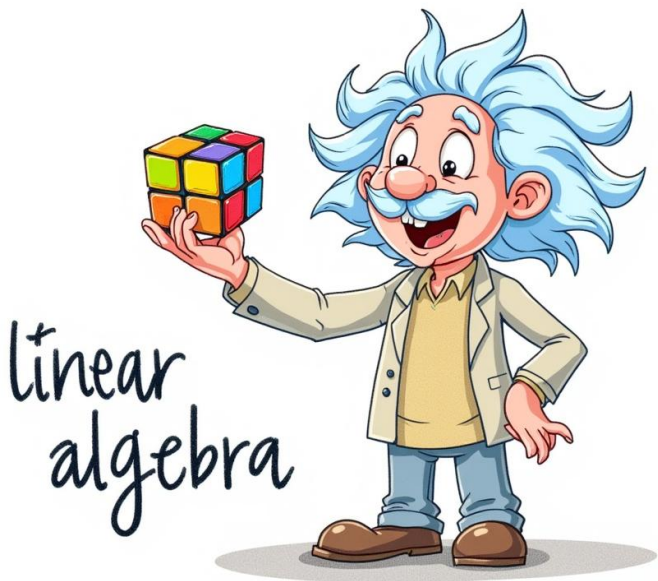


MLIR Part 4 - Linear Algebra in MLIR

Stephen Diehl



The Linalg dialect is a core idea in the MLIR ecosystem, it was built to make high-level tensor computations easier and more efficient. Essentially, it's all about simplifying the way we handle complex mathematical operations in a way that can be translated into fast, optimized code. At its core, the Linalg dialect uses a straightforward, declarative approach. This means it lays out operations and transformations in a way that keeps the math intact while also allowing for a variety of optimizations. It provides high-level linear algebra operations like pointwise calculations, matrix multiplications, and convolutions as single operations.

`linalg.generic`

The most generic operation in the Linalg dialect is `linalg.generic`. This operation allows us to define a generic tensor manipulation operations over indices. It is the most generic operation and all other operations are defined as specializations of this operation. You probably won't use this directly much, but it's useful to discuss first because it's the most fundamental operation.

```
#map_1d_identity = affine_map<(d0) -> (d0)>

func.func @add_tensors(%arg0: tensor<10xf32>, %arg1: tensor<10xf32>) -> tensor<10xf32> {
  %result = tensor.empty() : tensor<10xf32>
  %0 = linalg.generic {
    // attributes
    indexing_maps = [
      #map_1d_identity,
      #map_1d_identity,
      #map_1d_identity
    ],
    iterator_types = [
      "parallel"
    ]
  }
  // input tensors
  ins(%arg0, %arg1 : tensor<10xf32>, tensor<10xf32>)
  // output tensor
  outs(%result : tensor<10xf32>)
  // loop region
  {
    ^bb0(%arg2: f32, %arg3: f32, %arg4: f32):
      %1 = arith.addf %arg2, %arg3 : f32
      linalg.yield %1 : f32
    } -> tensor<10xf32>
  return %0 : tensor<10xf32>
}
```

Let's break down each component of this `linalg.generic` operation:

1. `indexing_maps`: This attribute defines how input and output tensors are accessed. Here we have 3 maps:

- First map for first input tensor (`%arg0`)
- Second map for second input tensor (`%arg1`)
- Third map for output tensor (`%result`)

All use `#map_1d_identity` which is defined as `affine_map<(d0) -> (d0)>`, meaning each element maps directly to the corresponding element (no coordinate transformations).

2. `iterator_types`: Specifies the type of iteration for each dimension. Here we have:

- `["parallel"]`: Single dimension marked as parallel, meaning iterations can be executed in any order or concurrently.

There are three possible iterator types in `linalg`:

- `parallel`: Iterations can be executed in any order or concurrently (like element-wise operations)
- `reduction`: Iterations contribute to a single result value (like summing along an axis)
- `window`: Iterations access a sliding window of elements (like convolutions)

3. `ins(%arg0, %arg1 : tensor<10xf32>, tensor<10xf32>):`

- Specifies the input tensors and their types
- Both are 1D tensors of 10 float32 elements

4. `outs(%result : tensor<10xf32>):`

- Specifies the output tensor and its type
- Also a 1D tensor of 10 float32 elements

5. The region (loop body):

```
{
^bb0(%arg2: f32, %arg3: f32, %arg4: f32):
  %1 = arith.addf %arg2, %arg3 : f32
  linalg.yield %1 : f32
}
```

- Takes scalar values from input tensors (`%arg2`, `%arg3`) and output tensor (`%arg4`)
- Performs floating point addition
- Yields result back using `linalg.yield`

This would perform the standard vectorized addition of two tensors element-wise, just like in NumPy:

```
np.array([1,2,3]) + np.array([2,3,4])
# array([3, 5, 7])
```

If this seems like a lot, don't worry it is. There are much simpler ways to write this that compile down to the same thing.

`linalg.map`

The same operation can be written as a generic `linalg.map` operation which abstracts away the precise details of the loop nest and index manipulations, instead directly applying the `arith.addf` operation to the input tensors.

```
module {
  func.func @addv(%arg0: tensor<10xf32>, %arg1: tensor<10xf32>, %out: memref<10xf32>) {
    %out2 = tensor.empty() : tensor<10xf32>
    %result = linalg.map { arith.addf } ins(%arg0, %arg1 : tensor<10xf32>, tensor<10xf32>) outs(%out2 : tensor<10xf32>)

    %l = bufferization.to_memref %result : tensor<10xf32> to memref<10xf32>
    memref.copy %l, %out : memref<10xf32> to memref<10xf32>

    return
  }
}
```

There are also named operations for common arithmetic map operations that are named specializations of the `linalg.map` operation.

- `linalg.add`
- `linalg.mul`
- `linalg.div`
- `linalg.sub`

linalg.reduce

The `linalg.reduce` operation is a powerful primitive that can be used to implement various reduction operations.

```
func.func @add_tensor() -> (f64) {
  %_ssa_0 = tensor.empty () : tensor<3xf64>
  %out = tensor.empty () : tensor<f64>
  %reduce = linalg.reduce { arith.addf } ins(%_ssa_0:tensor<3xf64>) outs(%out:tensor<f64>) dimensions = [0]
  %result = tensor.extract %reduce [] : tensor<f64>
  return %result : f64
}
```

In NumPy this operation can be written as:

```
import numpy as np
np.multiply.reduce([2,3,5]) # 30
```

linalg.matmul

We can also define a generic matrix multiplication operation in terms of direct tensor contractions. The matrix multiplication operation shown in the MLIR code can be written in conventional tensor notation as:

$$C_{ij} = \sum_{k=1}^{10} A_{ik} B_{kj}$$

Where:

- A is the 8×10 input matrix (memref<8x10xf32>)
- B is the 10×16 input matrix (memref<10x16xf32>)
- C is the 8×16 output matrix (memref<8x16xf32>)
- i ranges from 1 to 8
- j ranges from 1 to 16
- k is the summation index ranging from 1 to 10

This matches the MLIR implementation where:

- The `parallel` iterator types correspond to the free indices i and j
- The `reduction` iterator type corresponds to the summation over k
- The indexing maps $(i, j, k) \rightarrow (i, k)$, $(i, j, k) \rightarrow (k, j)$, and $(i, j, k) \rightarrow (i, j)$ correspond to accessing A_{ik} , B_{kj} , and C_{ij} respectively

```
func.func @matmul(
  %A: memref<8x10xf32>,
  %B: memref<10x16xf32>,
  %C: memref<8x16xf32>
) {
  linalg.generic {
    indexing_maps = [
      affine_map<(i, j, k) -> (i, k)>,
      affine_map<(i, j, k) -> (k, j)>,
      affine_map<(i, j, k) -> (i, j)>
    ],
    iterator_types = [
      "parallel",
      "parallel",
      "reduction"
    ]
  } ins(%A, %B : memref<8x10xf32>, memref<10x16xf32>) outs(%C : memref<8x16xf32>) {
    ^bb0(%lhs_one: f32, %rhs_one: f32, %init_one: f32):
      %tmp0 = arith.mulf %lhs_one, %rhs_one : f32
      %tmp1 = arith.addf %init_one, %tmp0 : f32
      linalg.yield %tmp1 : f32
  }
  return
}
```

Or in the much more succinct form using `linalg.matmul` operation.

```
func.func @matmul(
  %A: memref<8x10xf32>,
  %B: memref<10x16xf32>,
  %C: memref<8x16xf32>
) {
  linalg.matmul ins(%A, %B : memref<8x10xf32>, memref<10x16xf32>) outs(%C : memref<8x16xf32>)
  return
}
```

The tensor operations are then lowered into a sequence of loops using either the `affine.for` or `scf.for` for operations using the `convert-linalg-to-loops` or `convert-linalg-to-affine-loops` passes.

```
func.func @matmul(%arg0: memref<8x10xf32>, %arg1: memref<10x16xf32>, %arg2: memref<8x16xf32>) {
  affine.for %arg3 = 0 to 8 {
    affine.for %arg4 = 0 to 16 {
      affine.for %arg5 = 0 to 10 {
        %0 = affine.load %arg0[%arg3, %arg5] : memref<8x10xf32>
        %1 = affine.load %arg1[%arg5, %arg4] : memref<10x16xf32>
        %2 = affine.load %arg2[%arg3, %arg4] : memref<8x16xf32>
        %3 = arith.mulf %0, %1 : f32
        %4 = arith.addf %2, %3 : f32
        affine.store %4, %arg2[%arg3, %arg4] : memref<8x16xf32>
      }
    }
  }
  return
}
```

```
func.func @matmul(%arg0: memref<8x10xf32>, %arg1: memref<10x16xf32>, %arg2: memref<8x16xf32>) {
  %c0 = arith.constant 0 : index
  %c8 = arith.constant 8 : index
  %c1 = arith.constant 1 : index
  %c16 = arith.constant 16 : index
  %c10 = arith.constant 10 : index
  scf.for %arg3 = %c0 to %c8 step %c1 {
    scf.for %arg4 = %c0 to %c16 step %c1 {
      scf.for %arg5 = %c0 to %c10 step %c1 {
        %0 = memref.load %arg0[%arg3, %arg5] : memref<8x10xf32>
        %1 = memref.load %arg1[%arg5, %arg4] : memref<10x16xf32>
        %2 = memref.load %arg2[%arg3, %arg4] : memref<8x16xf32>
        %3 = arith.mulf %0, %1 : f32
        %4 = arith.addf %2, %3 : f32
        memref.store %4, %arg2[%arg3, %arg4] : memref<8x16xf32>
      }
    }
  }
  return
}
```

Broadcasting

Broadcasting in MLIR's Linalg dialect allows tensors of different shapes to be combined in operations, similar to NumPy's broadcasting rules. The smaller tensor is implicitly expanded to match the shape of the larger tensor along broadcast dimensions. This is particularly useful for operations like adding a vector to each row/column of a matrix or applying scalar operations to tensors.

Let's look at some common broadcasting patterns:

```
func.func @broadcast_1d_to_2d(%arg0: tensor<3xf32>) -> tensor<3x3xf32> {
  // Create an empty output tensor
  %init = tensor.empty() : tensor<3x3xf32>

  // Broadcast the input tensor along dimension 1 (columns)
  %result = linalg.broadcast
    ins(%arg0: tensor<3xf32>)
    outs(%init: tensor<3x3xf32>)
    dimensions = [1]

  return %result : tensor<3x3xf32>
}
```

In NumPy this operation can be written as:

```
arg0 = np.array([1,2,3])
result = np.broadcast_to(arg0, (3,3))
# array([[1, 2, 3],
#        [1, 2, 3],
#        [1, 2, 3]])
```

We can also combine broadcasting with other operations. For example, adding a vector to each row of a matrix:

```
func.func @add_vector_to_matrix(
  %matrix: tensor<3x4xf32>,
  %vector: tensor<4xf32>
) -> tensor<3x4xf32> {
  %init = tensor.empty() : tensor<3x4xf32>

  // First broadcast the vector
  %broadcasted = linalg.broadcast
```

```

    ins(%vector: tensor<4xf32>)
    outs(%init: tensor<3x4xf32>)
    dimensions = [0]

// Then add the broadcasted result to the matrix
%result = linalg.map { arith.addf }
    ins(%matrix, %broadcasted: tensor<3x4xf32>, tensor<3x4xf32>)
    outs(%init: tensor<3x4xf32>)

return %result : tensor<3x4xf32>
}

```

This is the equivalent to the common NumPy operation:

```

matrix = np.ones((3,4))
vector = np.array([1,2,3,4])
result = matrix + vector
# array([[2., 3., 4., 5.],
#        [2., 3., 4., 5.],
#        [2., 3., 4., 5.]])

```

Kernel Fusion

Kernel fusion is an important optimization technique that combines multiple operations into a single operation, reducing memory bandwidth requirements and improving performance. The Linalg dialect provides built-in support for fusing elementwise operations through its optimization passes.

Let's look at a simple example where we have two operations - addition followed by multiplication - that we want to fuse together. First, we'll write these as separate operations:

```

module {
  func.func @addmul(%arg0: tensor<10xf32>, %arg1: tensor<10xf32>, %arg2: tensor<10xf32>) -> tensor<10xf32> {
    // First create an empty tensor for intermediate results
    %0 = tensor.empty() : tensor<10xf32>

    // Add operation
    %1 = linalg.add ins(%arg0, %arg1 : tensor<10xf32>, tensor<10xf32>)
        outs(%0 : tensor<10xf32>) -> tensor<10xf32>

    // Create another empty tensor for the multiplication result
    %2 = tensor.empty() : tensor<10xf32>

    // Multiply operation
    %3 = linalg.mul ins(%1, %arg2 : tensor<10xf32>, tensor<10xf32>)
        outs(%2 : tensor<10xf32>) -> tensor<10xf32>

    return %3 : tensor<10xf32>
  }
}

```

Here we have two distinct operations: `linalg.add` followed by `linalg.mul`. Each operation requires reading from memory and writing back to memory, which can be inefficient.

Through MLIR's optimization passes, we can fuse these operations into a single kernel that performs both operations in one pass over the data:

```

#map = affine_map<(d0) -> (d0)>
module {
  func.func @addmul(%arg0: tensor<10xf32>, %arg1: tensor<10xf32>, %arg2: tensor<10xf32>) -> tensor<10xf32> {
    %0 = tensor.empty() : tensor<10xf32>
    %1 = linalg.generic {indexing_maps = [#map, #map, #map, #map],
        iterator_types = ["parallel"]} ins(%arg0, %arg1, %arg2 : tensor<10xf32>,
        tensor<10xf32>, tensor<10xf32>) outs(%0 : tensor<10xf32>) {
      ^bb0(%in: f32, %in_0: f32, %in_1: f32, %out: f32):
        %2 = arith.addf %in, %in_0 : f32
        %3 = arith.mulf %2, %in_1 : f32
        linalg.yield %3 : f32
      } -> tensor<10xf32>
    return %1 : tensor<10xf32>
  }
}

```

To get some intuition for how this works, let's look at the equivalent Python code:

```

# Unfused version - two separate loops
def unfused_addmul(a, b, c):
    # First loop - addition
    temp = []
    for i in range(len(a)):

```

```

        temp.append(a[i] + b[i])

# Second loop - multiplication
result = []
for i in range(len(temp)):
    result.append(temp[i] * c[i])

return result

# Fused version - single loop
def fused_addmul(a, b, c):
    result = []
    for i in range(len(a)):
        # Combine add and multiply in one loop iteration
        result.append((a[i] + b[i]) * c[i])
    return result

a = [1.0, 2.0, 3.0, 4.0, 5.0]
b = [1.0, 1.0, 1.0, 1.0, 1.0]
c = [2.0, 2.0, 2.0, 2.0, 2.0]

# Both produce same result but fused version only loops once
result = fused_addmul(a, b, c) # [(1+1)*2, (2+1)*2, (3+1)*2, ...]
```

The fused version combines both operations into a single `linalg.generic` operation that reads each input only once and writes the final result directly. This eliminates the intermediate tensor allocation and memory accesses.

To perform this fusion, we need to apply a specific sequence of optimization passes:

1. `--canonicalize`: Performs general cleanups and simplifications
2. `--linalg-fuse-elementwise-ops`: Fuse elementwise operations
3. `--cse`: Common Subexpression Elimination to clean up redundant operations
4. `--linalg-generalize-named-ops`: Converts named ops (like `linalg.add`) to generic form
5. `--convert-linalg-to-loops`: Converts the fused operations to explicit loops

We can run these passes and output the fused `addmul` operation to `fused_ops.mlir`, reducing the time complexity from two loops over n to a single loop over n .

```

mlir-opt separate_ops.mlir \
  --canonicalize \
  --linalg-fuse-elementwise-ops \
  --cse \
  --linalg-generalize-named-ops \
  --linalg-fuse-elementwise-ops \
  --convert-linalg-to-loops \
  -o fused_ops.mlir
```

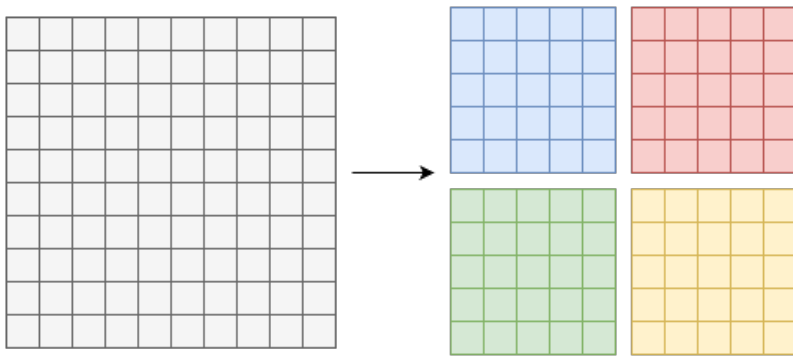
This sequence of passes ensures that operations are properly fused and then lowered to a form that can be efficiently executed. The fusion optimization is particularly important in deep learning workloads where chains of elementwise operations are common.

Tiling

Loop tiling is a crucial optimization technique that improves memory access patterns and cache utilization. In modern computer architectures, memory access is often the bottleneck rather than computation. When processing large matrices or tensors, if we access memory in a linear fashion, we may constantly need to fetch new data from main memory, which is much slower than cache memory.

Tiling breaks down large computations into smaller blocks that better fit into cache memory. For example, in matrix multiplication, instead of computing the entire row-column dot product at once (which might exceed cache size), we break it into smaller tiles that can reside in cache memory. This reduces cache misses and improves performance.

This optimization is particularly important for GPU architectures, where memory hierarchy and access patterns significantly impact performance. GPUs have a complex memory hierarchy including global memory, shared memory (similar to cache), and registers. By tiling computations to match the GPU's hardware characteristics—like shared memory size and warp size—we can dramatically improve performance. For instance, in CUDA programming, a common pattern is to load a tile of data into shared memory, synchronize threads, and then have each thread work on elements within that tile. More on this in the later section on GPU programming.



Here's an example of using MLIR to perform loop tiling on a matrix multiplication operation. We'll start with a simple tensor-based matmul and transform it into tiled affine loops.

```
module {
  func.func @matmul(%arg0: tensor<10x10xf32>, %arg1: tensor<10x10xf32>,
    %arg2: tensor<10x10xf32>) -> tensor<10x10xf32> {
    %0 = linalg.matmul
      ins(%arg0, %arg1 : tensor<10x10xf32>, tensor<10x10xf32>)
      outs(%arg2 : tensor<10x10xf32>) -> tensor<10x10xf32>
    return %0 : tensor<10x10xf32>
  }
}
```

To transform this into tiled loops, we need several transformation passes:

1. `--convert-tensor-to-linalg`: Converts tensor operations to linalg dialect
2. `--linalg-generalize-named-ops`: Converts named operations (like matmul) into generic linalg operations
3. `--one-shot-bufferize="bufferize-function-boundaries"`: Converts tensor operations to use buffers
4. `--buffer-deallocation-pipeline`: Handles buffer deallocation
5. `--convert-bufferization-to-memref`: Converts buffer operations to memref operations
6. `--convert-linalg-to-affine-loops`: Lowers linalg operations to affine loops
7. `--affine-loop-tile="tile-size=5"`: Tiles the affine loops with size 5
8. `--canonicalize --cse`: Cleanup passes

Here's the complete transformation pipeline:

```
mlir-opt tiling.mlir \
  --convert-tensor-to-linalg \
  --linalg-generalize-named-ops \
  --one-shot-bufferize="bufferize-function-boundaries" \
  --buffer-deallocation-pipeline \
  --convert-bufferization-to-memref \
  --convert-linalg-to-affine-loops \
  --affine-loop-tile="tile-size=5" \
  --canonicalize \
  --cse \
  -o tiling-transformed.mlir
```

This produces the following tiled implementation:

```
#map = affine_map<(d0) -> (d0)>
#map1 = affine_map<(d0) -> (d0 + 5)>
module {
  func.func @matmul(%arg0: memref<10x10xf32, strided<[?, ?], offset: ?>>, %arg1:
    memref<10x10xf32, strided<[?, ?], offset: ?>>, %arg2: memref<10x10xf32,
    strided<[?, ?], offset: ?>>) -> memref<10x10xf32, strided<[?, ?], offset: ?>>
  {
    affine.for %arg3 = 0 to 10 step 5 {
      affine.for %arg4 = 0 to 10 step 5 {
        affine.for %arg5 = 0 to 10 step 5 {
          affine.for %arg6 = #map(%arg3) to #map1(%arg3) {
            affine.for %arg7 = #map(%arg4) to #map1(%arg4) {
              affine.for %arg8 = #map(%arg5) to #map1(%arg5) {
                %0 = affine.load %arg0[%arg6, %arg8] : memref<10x10xf32, strided<[?, ?], offset: ?>>
                %1 = affine.load %arg1[%arg8, %arg7] : memref<10x10xf32, strided<[?, ?], offset: ?>>
                %2 = affine.load %arg2[%arg6, %arg7] : memref<10x10xf32, strided<[?, ?], offset: ?>>
                %3 = arith.mulf %0, %1 : f32
                %4 = arith.addf %2, %3 : f32
                affine.store %4, %arg2[%arg6, %arg7] : memref<10x10xf32, strided<[?, ?], offset: ?>>
              }
            }
          }
        }
      }
    }
  }
}
```

```

%alloc = memref.alloc() : memref<10x10xf32>
%cast = memref.cast %alloc : memref<10x10xf32> to memref<10x10xf32, strided<[?, ?], offset: ?>>
memref.copy %arg2, %alloc : memref<10x10xf32, strided<[?, ?], offset: ?>> to memref<10x10xf32>
return %cast : memref<10x10xf32, strided<[?, ?], offset: ?>>
}
}

```

The tiled output shows the original 10x10 matrix multiplication broken into smaller 5x5 tiles, which can help improve cache utilization and locality of memory accesses.

External References

- [MLIR: Linalg Dialect](#)
- [MLIR Open Meeting 2022-01-27: Introduction to Linalg.generic](#)
- [2024 EuroLLVM - MLIR Linalg Op Fusion - Theory & Practice](#)
- [NumPy Broadcasting](#)
- [MLIR Codegen Dialects for Machine Learning Compilers](#)
- [MLIR Linalg Dialect and Patterns](#)
- [Linalg Dialect Rationale: The Case For Compiler-Friendly Custom Operations](#)
- [Anatomy of Linalg.generic](#)
- [IREE/MLIR/Linalg tutorial](#)